IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received October 20, 2019, accepted November 12, 2019, date of publication November 22, 2019,
date of current version December 10, 2019.

Digital Object Identifier 10.1109/ACCESS.2019.2955086

Grouped SMOTE With Noise Filtering Mechanism for Classifying Imbalanced Data

KE CHENG[1], CHEN ZHANG[1], HUALONG YU[1,2], XIBEI YANG[1], HAITAO ZOU[1,2], AND SHANG GAO[1,2]

[1]School of Computer, Jiangsu University of Science and Technology, Zhenjiang 212003, China
[2]Artificial Intelligence Key Laboratory of Sichuan Province, Sichuan University of Science and Engineering, Yibin 644000, China

Corresponding author: Shang Gao (gao_shang1972@163.com)

This work was supported in part by the Natural Science Foundation of Jiangsu Province of China under Grant BK20191457, in part by the Open Project of Artificial Intelligence Key Laboratory of Sichuan Province under Grant 2019RYJ02, in part by the National Natural Science Foundation of China under Grant 61305058 and Grant 61572242, in part by the China Postdoctoral Science Foundation under Grant 2013M540404 and Grant 2015T80481, in part by the Jiangsu Planned Projects for Postdoctoral Research Funds under Grant 1401037B, and in part by the Qing Lan Project of Jiangsu Province of China.

ABSTRACT SMOTE (Synthetic Minority Oversampling TEchnique) is one of the most popular and well-known sampling algorithms for addressing class imbalance learning problem. The merits of SMOTE reflect at that in comparison with the random oversampling technique, it can alleviate the problem of overfitting to a large extent. However, two drawbacks of SMOTE have also been observed as follows, 1) it tends to propagate the noisy information in the procedure of oversampling; 2) it always assigns a global neighborhood parameter K but neglects the local distribution characteristics. To synchronously deal with these two problems, a grouped SMOTE algorithm with noise filtering mechanism (GSMOTE-NFM) is presented in this article. The algorithm firstly adopts Gaussian-Mixture Model (GMM) to explore the real distributions of the majority and minority classes, respectively. Then, most noisy instances can be removed by comparing the probability densities of the same instance in two different classes. Next, two new GMMs are constructed on the rest majority and minority class instances, respectively. Furthermore, all minority class instances can be divided into three different groups: safety, boundary and outlier, based on the corresponding probability density information. Finally, we assign an individual parameter K to the instances belonging to each specific group to generate new instances. We tested GSMOTE-NFM algorithm on 24 benchmark binary-class data sets with three popular classification models, and compared it with several state-of-the-art oversampling algorithms. The results indicate that our algorithm is significantly superior than the original SMOTE algorithm and several SMOTE-based modified methods.

INDEX TERMS Sampling, class imbalance learning, SMOTE, Gaussian-Mixture model, probability density.

I. INTRODUCTION

In recent years, class imbalance learning (CIL) problem has become a challenging problem in the field of machine learning and data mining [1], [2]. As a hotspot, it has also attracted a significant amount of interests from some leading and important academic journals and conferences. In fact, *AAAI*'00 [3], *ICML*'03 [4], *PAKDD*'09 [5] and *IJCAI*'17 [6] have hosted the specific workshops to discuss CIL problem, as well *ACM SIGKDD Explorations Newsletter* [7], *Neurocomputing* and *IEEE Transactions on Knowledge and Data Engineering* have also organized some special issues about CIL. In *ICDM*'05, CIL problem has also been listed into the Top 10 challenging problems in data mining [8]. During the past two decades, CIL problem has been observed in a lot of real-world applications, including fraud detection [9], [10], network intrusion detection [11], [12], disease diagnosis [13], [14], software defect detection [15], [16], industrial manufacturing [17], Bioinformatics [18], [19], environment resource management [20], and security management [21], etc.

To deal with the CIL problem, sampling [22]–[24], cost-sensitive learning [25]–[28], threshold strategy [29]–[31] and ensemble learning [32]–[35] are the most frequently used techniques. In contrast with the other CIL techniques, sampling strategy possesses two remarkable advantages as

The associate editor coordinating the review of this manuscript and approving it for publication was Rajesh Kumar.

follows, 1) it is easier to be implemented than several other CIL techniques as it only changes the instance distribution; 2) it is independent of the underlying classification model. Therefore, we focus on solving CIL problem by the sampling technique in this paper.

Sampling, as its name indicates, means that it needs to readjust the distribution of the training set to make it balance, i.e., the number of instances belonging to different classes are almost same, by either reducing the majority class instances or increasing the minority class instances. The former is called undersampling, while the latter is named oversampling. Random undersampling (RUS) and random oversampling (ROS) are the most popular and simplest sampling algorithms. The former randomly throws out the instances belonging to the majority class, while the latter randomly duplicates the instances belonging to the minority class. It is possible for RUS to remove some instances which contain the important classification information, and ROS tends to make the classification model overfit. Despite both undersampling and oversampling have their pros and cons, several studies have reported that there is a less risk to use oversampling than undersampling in most real-world applications [30], [31], [36], [37].

Among all oversampling strategies, the SMOTE algorithm [23] is clearly the most popular and widely used one. The SMOTE algorithm overcomes the drawback of ROS which is easier to make the classifier overfit, by inserting the synthetic minority-class instance between two randomly adjacent minority-class examples. Although SMOTE has showed its superiority compared with ROS, it has several other defects as follows, 1) it tends to propagate the noisy information during sampling; 2) it always assigns a global neighborhood parameter K but neglects the local distribution characteristics.

To deal with the drawbacks of SMOTE which are listed above, some improved SMOTE algorithms have been proposed. Han *et al.*, [38] emphasized that the instances existing around the boundary are more significant than the others, thereby proposed the Borderline SMOTE (B-SMOTE) algorithm. The B-SMOTE algorithm firstly puts all minority-class instances into three nonoverlapping groups including safety, danger and noise, and then only conducts SMOTE algorithm in danger group. In most cases, the instances existing in danger group distribute nearby the borderline. They also divide B-SMOTE into two different versions, B-SMOTE1 and B-SMOTE2, where the former only generates new instances between the borderline minority-class instances and their nearest neighbors belonging to the same class, while the latter produces the synthetic examples by adding the majority-class instances into the neighborhood calculation. The merits of the B-SMOTE algorithm lay in that it can effectively filter the noise appearing in the minority-class, as well enlarge the decision region of the minority-class. However, we have to note its drawback, i.e., the neighborhood calculation is based on the Euclidean distance, which runs a risk to accurately divide the samples into the different groups when the class imbalance ratio is relatively high. Bunkhumpornpat *et al.*, [39] proposed Safe-Level-SMOTE (SL-SMOTE) algorithm, which is similar with B-SMOTE. In SL-SMOTE, each minority-class instance is assigned a safe level coefficient by counting the number of minority-class instances in its K nearest neighbors. Then according to the safe level coefficient, SL-SMOTE can filter noise and generate the synthetic minority-class instances which are more close to the safe region belonging to the minority class. Similar to B-SMOTE, SL-SMOTE can effectively alleviate the impact of noise, but need to take the risk of inaccurate estimation of instance region, too. SMOTE-IPF is a more ingenious sampling algorithm with considering to filter the noisy examples [40]. It firstly divides the training set into n nonoverlapping subsets, and then trains one C4.5 decision tree model on each subset, finally for each training instance, uses the result of ensemble voting to judge whether it is a noisy example, and to remove it if it has been estimated as a noise. It is obvious that the noise filtering mechanism adopted by the SMOTE-IPF is more robust than that of the Borderline-SMOTE and SL-SMOTE as it discards the neighborhood calculation. However, SMOTE-IPF fails to assign the individual parameters for different instances by considering their distribution characteristics. Garcia *et al.*, [41] also observed the defect of Euclidean neighborhood, thus proposed the surrounding neighborhood-based SMOTE (SN-SMOTE). In their work, three different kinds of neighborhood are defined, namely nearest centroid neighborhood (NCN), Gabriel graph (GG) and relative neighborhood graph (RNG), respectively. Although SN-SMOTE does not group the instances according to their distribution characteristics, the neighborhoods adopted by it take into account both proximity and the spatial distribution of the examples. Therefore, we can say the SN-SMOTE uses an individual sampling strategy. Table 1 summarizes the characteristics of the SMOTE and several SMOTE-based modification algorithms in terms of both noise filter mechanism and individual sampling strategy.

TABLE 1. Summaries Of SMOTE and several SMOTE-based modification algorithms in two important mechanisms.

Algorithm	Filtering Noise?	Individual Sampling?
SMOTE	None	None
B-SMOTE	Partial	Partial
SL-SMOTE	Partial	Great
SMOTE-IPF	Perfect	None
SN-SMOTE	None	Great

For each mechanism, the adopted strategy can be labeled as one of the following feature words, None, Partial, Great and Perfect, where None<Partial<Great<Perfect.

Considering that the existing SMOTE-based algorithms hold some imperfect designs more or less, a novel SMOTE modification algorithm is presented in this article. Specifically, to get rid of the limitation of unstable noise verification and group estimation, we adopt Gaussian-Mixture Model (GMM) to explore the prior distribution of the data set. The advantage of adopting GMM lies in that it can

approximate any distribution type, even including those irregular distributions, further providing the accurate estimation about the instance position information. In our proposed GSMOTE-NFM algorithm, the noisy instances can be firstly found and filtered by comparing the probability densities of each instance existing in two different classes. Then, we continue to construct GMMs on the filtered data, and use the comparison relationship of the probability densities to divide all instances into three different groups which can reflect the local position information of each instance well. Finally, we assign the specific parameter for the instances of each group according to their characteristics to implement SMOTE. To our best knowledge, no previous work use probability density estimation strategy to improve SMOTE algorithm. We believe that it has the relatively perfect noise filter and individual sampling mechanisms. We tested GSMOTE-NFM algorithm on 24 benchmark binary-class imbalanced data sets with three popular classification models, and compared it with random oversampling (ROS), SMOTE and several SMOTE-based modification algorithms. The results indicate that our algorithm is significantly superior than the other methods.

The rest of this article is organized as follows. Section II briefly introduces some preliminaries, including SMOTE and its problem description. In Section III, we describe the proposed algorithm in detail. Section IV provides the experimental results and analysis. Section V concludes the contribution of this paper, as well indicates its limitations.

II. PRELIMINARIES

A. SMOTE

As mentioned in Section I, SMOTE [23] is the most popular and widely used oversampling algorithm. It generates new artificial minority-class instances by interpolating among several minority-class examples that lie together. Specifically, for a selected minority-class instance x, SMOTE firstly find its K nearest neighbors $x_1, x_2, \ldots, x_K$ in the minority class. Then, one instance x_i is randomly extracted from these K nearest neighbors. Next, a random number r laying between (0, 1) is generated to multiply the difference between x and x_i, i.e., $r \times (x_i - x)$. Finally, a new artificial minority-class instance x' can be generated by,

$$x' = x + r \times (x_i - x) \tag{1}$$

That means in SMOTE, each new synthetic minority-class instance will locate on a random position of the line connected two original minority-class instances.

Fig.1 gives an example to explain how to generate new synthetic instances in SMOTE.

B. PROBLEM DESCRIPTION

Although SMOTE can alleviate the overfitting phenomenon of random oversampling by interpolating synthetic instances on new positions, it still has two drawbacks. The first one is that it can propagate the noisy information which causes some new instances appearing on inappropriate positions.

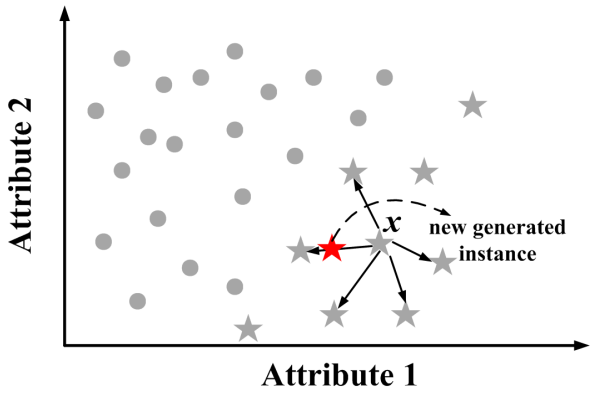

FIGURE 1. An example of generating new synthetic instances by SMOTE, where x denotes the selected minority-class instance, ★ and ● respectively denote the original minority-class and majority-class instances, and ★ denotes the new generated minority-class instance.

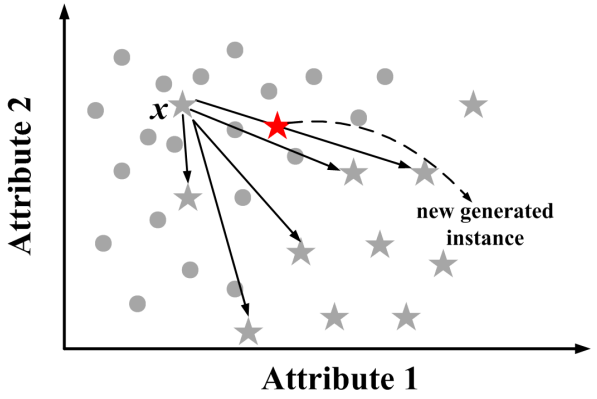

FIGURE 2. An example of noise propagation in SMOTE, where x denotes the selected minority-class instance, ★ and ● respectively denote the original minority-class and majority-class instances, and ★ denotes the new generated minority-class instance.

The second one is that in SMOTE, all instances share the same neighborhood parameter K, but neglect the distribution characteristics. Fig.2 presents how to propagate the noisy information in SMOTE, while in Fig.3, the significance of individual sampling is highlighted.

In Fig.2, we can observe that in SMOTE, the minority-class noisy instances can decrease the rationality of sampling. It is even possible to generate the synthetic minority-class instances in the majority-class regions which haven't minority-class instances originally. Therefore, we can say that the noisy instances are extremely harmful for SMOTE.

In Fig.3, it is not difficult to observe that except the noisy instances, the other examples can be roughly divided into three different types as following, safety, boundary and outlier, according to their distribution characteristics. Generally, the instances in the group of safety has very high probability density in the class itself, but quite low probability density in the other class, i.e., its probability density in the class itself is much larger than that in the other class; the instances in the group of boundary have the medium probability densities in both classes and the difference of the probability densities between two classes is quite small; the instances belonging

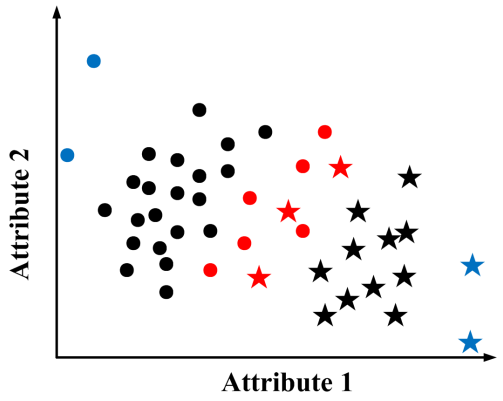

FIGURE 3. An example of grouping instances based on their position information, the star and circle denote the minority-class and majority-class instances, respectively. Specifically, the instances labeled as black belong to the safety group, red instances represent the borderline ones, while the blue instances denote the outliers.

to the group of outlier have very low probability densities in both classes, as well it generally has a little larger probability density in the class itself than that in the other class; while the noisy instances have the totally inverse characteristics in comparison with that in safety, i.e., they have high probability densities in the other class, but a significantly lower probability densities in the class itself. For the instances in different regions, adopting the same neighborhood parameter K may destroy SMOTE. For examples, assigning a small K for the safety instances may decrease the diversity of new generation instances; giving a large K for the boundary instances may decrease the probability of generating the examples surrounding the borderline; while designating a large K for the outlier can produce a similar risk as reserving the noise, i.e., generating new unexpected outlier instances.

Based on the problem description above, we have to admit that the affection of data distribution for SMOTE is quite sophisticated. Therefore, it is necessary to first explore the data prior distribution in depth, then to combine the data distribution information to refine the sampling procedure of SMOTE.

III. METHODS

A. GAUSSIAN-MIXTURE MODEL

Gaussian Mixture Model (GMM) models the density of a d-dimensional random variable x as a weighted sum of some Gaussian densities,

$$f(x) = \sum_{m=1}^{M} \pi_m \phi(x; \mu_m, \Sigma_m) \tag{2}$$

where $\phi(x; \mu_m, \Sigma_m)$ is a Gaussian density with mean vector μ_m and covariance matrix $\sum m$, i.e.,

$$\phi(x; \mu_m, \Sigma_m) = \frac{1}{\sqrt{(2\pi)^{d/2} |\Sigma_m|^{1/2}}} \exp \times \left[-\frac{1}{2}(x - \mu_m)^{\mathrm{T}} \Sigma_m^{-1} (x - \mu_m) \right] \tag{3}$$

and π_m are the positive mixing weights or mixing probabilities that satisfy the constraint $\sum_{m=1}^{M} \pi_m = 1$. In theory, using GMM can approximate any probability distribution, when and only when the number of the parameter M is set suitably. That is to say, GMM can approximately reflect the real probability density of an instance in any one specific data set.

Given a data set including N instances, to accurately model the GMM, we firstly need to solve lots of unknown parameters, i.e., $\theta = \{\mu_1, \Sigma_1, \ldots, \mu_M, \Sigma_M, \pi_1, \ldots, \pi_M\}$. It is difficult to directly address this problem. However, it can be transformed to be an expected log-likelihood function as below,

$$\begin{aligned} \mathrm{E}(l(\theta)) &= \mathrm{E}\left\{ \log \prod_{i=1}^{N} f(x_i; \theta) \right\} \\ &= \mathrm{E}\left\{ \sum_{i=1}^{N} \log \left(\sum_{m=1}^{M} \pi_m \phi(x_i; \mu_m, \Sigma_m) \right) \right\} \end{aligned} \tag{4}$$

where $l(\theta)$ is the log-likelihood of the whole data set. Then all parameters can be iteratively solved by EM algorithm [42].

In this work, GMM is used to accurately estimate the probability density of each instance, further to eliminate noisy ones and group the others. As a key parameter, M, i.e., the number of Gaussian models in GMM, we know if it is inappropriately set, the estimation can be overfitting or underfitting. Therefore, without loss of generality, we empirically designate it as $M = 5$.

B. NOISE FILTER MECHANISM

As mentioned in Section II, the noise reflects in that it has a significantly higher probability density in the other class than in the class itself. Therefore, we intend to find them by comparing the probability densities of each instance in the class itself and the other class, and then to remove them from the original data set.

Given a binary-class data set with N instances, for the ith instance, suppose its probability density acquired from the GMM belonging to the same class is P_i, and that in the other class is P'_i, then it can be estimated as a noisy instance when and only when $P_i < \lambda \times P'_i$, where λ is a default parameter which lies in (0,1]. In this paper, the parameter λ is empirically set to be 0.5.

The procedure of noise filtering can be briefly described as below,

Input: a binary-class skewed and labeled data set $D = \{(x_1, y_1), \ldots, (x_N, y_N)\}$, the parameter λ which lies in (0, 1], the number of Gaussian models in GMM M

Output: a filtered data set D'

Procedure:

1) divide D into D^+ and D^- based on the label of each instance, where D^+ and D^- indicate the minority class and majority class sets, respectively;
2) construct GMM$^+$ and GMM$^-$ respectively based on the data sets D^+ and D^-, and the parameter M;

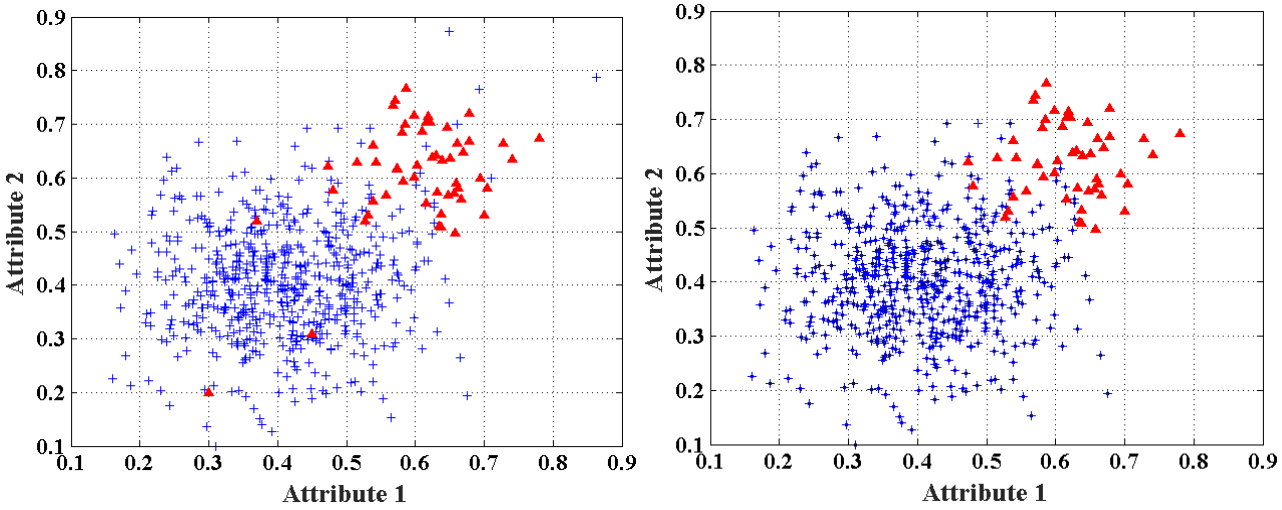

FIGURE 4. An example of noise filter on a synthetic two dimensional binary-class imbalanced data set, where the left subgragh indicates the data distribution before noise filter, and the right subgraph denotes the data distribution after noise filter, as well ▲ corresponds the minority class instances and + represents the majority class instances.

3) for each instance $x_i \in D$, calculate respectively its probability density in the same class p_i and that in the other class p_i' by using GMM$^+$ and GMM$^-$, if $P_i < \lambda \times P_i'$, then remove the instance from D, i.e., $D = D - \{x_i\}$;
4) repeat the procedure until all instances in D have been detected;
5) $D' = D$.

A simple example of noise filter using the algorithm above on a synthetic two dimensional binary-class imbalanced data set is presented in Fig. 4. By comparing the two subgraphs in Fig. 4, it can be observed that no matter for the majority class, or the minority class, the noisy instances have been removed, which shows the proposed noise filter algorithm is effective and feasible.

C. INDIVIDUAL SAMPLING

Next, a grouped strategy of the minority class instances will be designed according to the data distribution. As discussed in Section II, we can roughly divide the instances into three different groups: boundary, safety and outlier. Here, we used two parameters λ and η to divide the groups, where the parameter λ has been used to filter the noisy instances, and η is a small positive number lying in [0, 1]. For a specific minority class instance x_i, the division rule is presented as below,

$$\begin{cases} x_i \in \text{outlier}, & \text{if } p_{i+} < \mathrm{T}_+ || p_{i-} < \mathrm{T}_- || p_{i+} > p_{i-} \\ x_i \in \text{boundary}, & \text{if } p_{i+} < {}^{P_{i-}}\!/_{\lambda} \\ x_i \in \text{safety}, & \text{others} \end{cases} \tag{5}$$

where p_{i+} and p_{i-} respectively denote the probability densities of the minority class instance x_i acquired from GMM$^+$ and GMM$^-$, while T_+ and T_- respectively denote the outlier thresholds in the minority class and the majority class which can be acquired from,

$$\begin{aligned} \mathrm{T}_+ &= \tilde{p}_{\lfloor \eta \times |D'^+| \rfloor} \\ \mathrm{T}_- &= \tilde{p}_{\lfloor \eta \times |D'^-| \rfloor} \end{aligned} \tag{6}$$

where $|D'^+|$ and $|D'^-|$ respectively denote the number of instances in the minority class and the majority class, $\tilde{p}$ denotes the ascending sort of P. As η is a relatively small positive number between 0 and 1, that means only a few instances can be seen as the outliers, which is consistent with the reality. In this paper, the parameter η is empirically set to be 0.1 according to the feedback of lots of experimental results. That is to say, no more than 10% minority instances are regarded as the outliers. However, as the constraint of three parallel conditions, the number of outliers is generally far less than 10% minority class instances.

After division of outliers, the second decision condition in Eq.(5) would be used to find the instances around the boundary. It can be clearly observed that the condition is totally invertible in comparison with the noise decision condition. After that, all remaining instances can be put into the group of safety.

When sampling with SMOTE, the instances in different groups should be dealt discriminatively. By observing Fig.3, it is not difficult to find that the instances in the group of safety has generally far more compact than that in two other groups, hence a relatively large neighborhood parameter K can be safely allocated for the instances in the group of safety. In this paper, a commonly used value, i.e., $K = 5$, is designated for the instances in the safety group. While for the instances in the group of boundary which generally has a relatively sparse distribution, to decrease the risk that the new generated instances are far from the borderline, K should be designated a relatively smaller value than that corresponding to the safety group. In this paper, $K = 3$ is designated for the instances

belonging to the group of boundary. As for the outliers, we know that they are usually far from the other instances. Therefore, to avoid that the new generated instances appear at the unpredictable positions, K is assigned as 1, as well the Eq. (1) is rewritten as,

$$x' = x + \frac{1}{2} \times (1 + r) \times (x_{\text{nearest}} - x) \tag{7}$$

where x_{nearest} is the nearest instance to the outlier x.

The individual sampling procedure is described as below,

Input: a binary-class skewed and labeled data set $D' = \{(x_1, y_1), \ldots, (x_L, y_L)\}$ without noise, the parameter η which lies in [0, 1], the parameter λ which lies in (0,1], the number of Gaussian models in GMM M

Output: the final sampling set D''

Procedure:

1) divide D' into D'^{+} and D'^{-} based on the label of each instance, where D'^{+} and D'^{-} indicate the minority class and majority class sets, respectively;
2) construct GMM^{+} and GMM^{-} respectively based on the data sets D'^{+} and D'^{-}, and the parameter M;
3) divide the instances in D'^{+} into three different groups: D_{outliers}, D_{boundary} and D_{safety} according to the Eq. (5);
4) extract an instance $x_i \in D'^{+}$ randomly,
   If $x_i \in D_{\text{safety}}$
      set $K = 5$;
      find K nearest neighbors of x_i in D'^{+};
      generate a new instance x' by using Eq. (1);
   Else if $x_i \in D_{\text{boundary}}$
      set $K = 3$;
      find K nearest neighbors of x_i in D'^{+};
      generate a new instance by using Eq. (1);
   Else
      set $K = 1$;
      find the nearest neighbor of x_i in D'^{+};
      generate a new instance x' by using Eq. (7);
   End
5) $D' = D' \cup x'$;
6) repeat the step (4) and the step (5) until satisfying the stopping condition;
7) $D'' = D'$.

In the procedure of the individual sampling described above, the stopping condition in the step (6) generally associates with the pre-defined sampling rate. In this work, we adopt $|D'^{-}| - |D'^{+}|$ as the sampling rate to guarantee that the final training set D'' is totally balanced.

D. THE PROPOSED ALGORITHM

The proposed GSMOTE-NFM algorithm integrates the procedure of noise filter and the procedure of individual sampling, and its procedure is briefly described as below,

Input: a binary-class skewed and labeled data set $D = \{(x_1, y_1), \ldots, (x_N, y_N)\}$, the parameter λ which lies in (0, 1], the parameter η which lies in [0, 1], the number of Gaussian models in GMM M

Output: the final training set D''

Procedure:

1) call the noise filter algorithm to get a reduced set D' from the original data set D;
2) call the individual sampling algorithm to get the final training set D'' from the reduced set D'.

On the final training set D'', any one classification algorithm can be used and be expected to acquire an excellent enough classification performance.

IV. EXPERIMENTS AND DISCUSSIONS

A. DATA SETS DESCRIPTION

We collected 24 binary-class imbalanced data sets to validate the effectiveness of the proposed GSMOTE-NFM algorithm. The collection includes 3 data sets from UCI machine learning repository [43], i.e., ILPD, seeds_2_vs _1_3, wilt, and the remaining 21 data sets from Keel data repository [44]. Specifically, these data sets have different number of attributes, number of instances and class imbalance ratios (the ratio between the number of the majority class instances and the number of the minority class instances). The detailed information about these data sets are provided in Table 2.

B. EXPERIMENTAL SETTINGS

To validate the effectiveness and superiority of the proposed GSMOTE-NFM algorithm, we compared it with lots of state-of-the-art algorithms, including baseline algorithm without sampling (Baseline), random oversampling (ROS), SMOTE [23], B-SMOTE1 [38], B-SMOTE2 [38], SL-SMOTE [39], SMOTE-IPF [40], GG-SMOTE [41], RNG-SMOTE [41] and NCN-SMOTE [41]. Furthermore, to guarantee the impartiality of the comparative experiments, all SMOTE-based algorithms adopt a uniform neighborhood parameter $K = 5$ and a uniform sampling rate to guarantee the absolute balance of the final training set. As for the other unique parameters in those comparative algorithms, we adopt the default ones recommended in the corresponding references.

To demonstrate our proposed sampling algorithm is irrelevant to a specific classifier, we randomly adopt three different classification algorithms to conduct the comparative experiments, which are Support Vector Machine (SVM) [45], Logistic Regression (LR) [46] and C4.5 Decision Tree [47], respectively. In particular, for SVM, the widely used Gaussian radial basis (RBF) Kernel function is adopted. In addition, for any one specific classifier, all sampling algorithms share the same parameters, which can guarantee the impartiality of the comparative experiments.

Before training any classifier, each data set is scaled into [0,1] interval. Also, considering for CIL problem, the classification accuracy is not an excellent performance evaluation metric any longer, hence we adopt *F-measure* and *G-mean* as the performance evaluation metrics. In particular, *F-measure* investigates the trade-off between *precision* and *recall*, while *G-mean* detects the trade-off between the accuracy of the majority class and that of the minority class.

TABLE 2. Data sets used in this paper.

Data set	#attributes	#instances	IR
glass0	9	214	2.1
glass1	9	214	1.8
glass_0_1_2_3_vs_4_5_6	9	214	3.2
new_thyroid1	5	215	5.1
new_thyroid2	5	215	5.1
vehicle1	18	846	2.9
vehicle2	18	846	2.9
vehicle3	18	846	3.0
pima	8	768	1.9
ecoli1	7	336	3.4
ecoli2	7	336	5.5
ecoli_0_3_4_vs_5	7	200	9.0
ecoli_0_1_vs_2_3_5	7	244	9.2
yeast_2_vs_4	8	514	9.1
yeast_0_3_5_9_vs_7_8	8	506	9.1
yeast_0_5_6_7_9_vs_4	8	528	9.4
abalone9_18	8	731	16.4
haberman	3	306	2.8
wisconsin	9	683	1.9
ILPD	10	583	2.5
seeds_2_vs_1_3	7	210	2.0
wilt	5	4839	17.5
vowel0	13	988	10.0
led7digit_0_2_4_5_6_7_8_9_vs_1	7	443	11.0

#attributes, #instances and IR denote the number of attributes, the number of instances, and the class imbalance ratio, respectively.

Finally, considering the randomness of the experiments, the experimental results might be unstable. Therefore, we adopt 10 times' random 5-fold cross validation to calculate the average result for each algorithm. All experimental results were given in the form of mean $\pm$ standard deviation.

C. RESULTS AND DISCUSSION

From Table 3 to Table 8, the comparative results of various algorithms with three different classifiers on 24 data sets are given in detail. Specifically, the best result in each data set has been highlighted in boldface.

By the results in these tables, we observe:

1) In comparison with Baseline algorithm which only uses the original training data, on most data sets, various oversampling algorithms can promote the classification performance more or less, which verifies the effectiveness of sampling technique for addressing CIL problem again.
2) On some data sets, Baseline algorithm acquires excellent enough classification performance, e.g., wisconsin, seeds_2_vs_1_3 and vowel0. We consider that the phenomenon associates with many distributions factors [30]. If the original distribution of a data set has a large classification margin, a few noise and outliers, the adoption of CIL techniques is indeed useless. In other words, CIL techniques can only help to improve the classification performance for those data sets which are destroyed by the skewed data distribution.
3) On most data sets, SMOTE doesn't produce significantly better classification performance than ROS algorithm. We believe that it is related with the noise and outlier propagation mechanism of SMOTE, causing the classification boundaries of some data sets emerge on some unpredictable locations.
4) As for the existing SMOTE-based modification algorithms, no matter the ones with the noise filter mechanism, with the individual sampling mechanism, or with both can produce better performance than SMOTE algorithm on most data sets, indicating that these two mechanisms are valuable for promoting the quality of the data distribution.
5) Although on the same data set, there exists the significant difference about the classification results when adopting different classifiers, we can still observe the superiority of the proposed GSMOTE-NFM algorithm. Specifically, in terms of F-measure metric, GSMOTE-NFM acquires the best results on 14, 7 and 8 data sets respectively with SVM, Logistical Regression and C4.5 decision tree classifiers, and in terms of G-mean metric, it produces 14,10 and 13 best results with three different classifiers, respectively. In comparison with B-SMOTE series algorithms, the proposed GSMOTE-NFM algorithm amends the defect that the accuracy of Euclidean calculation is apt to be impacted by the variation of distribution density. While in comparison with the other SMOTE-based modification algorithms, GSMOTE-NFM algorithm improves the quality of either noise filter, or individual parameter assignment, or both. That also verifies the correctness and reasonability of the idea in this paper.

D. STATISTICAL ANALYSIS

In order to present a thorough understanding of the comparison of various sampling algorithms, we also provide their statistical results. Firstly, Friedman test is used to detect

TABLE 3. F-measure of the comparative algorithms by using SVM classifier.

Data-set	Baseline	ROS	SMOTE	B-SMOTE1	B-SMOTE2	SL-SMOTE	GG-SMOTE	RNG-SMOTE	NCN-SMOTE	SMOTE-IPF	GSMOTE-NFM
glass0	.5673±.0134	.6884±.0143	.6949±.0089	.6931±.0090	.6914±.0179	.6731±.0207	.6623±.0118	.6592±.0096	.6734±.0168	.6693±.0142	**.6969±.0075**
glass1	.5685±.0286	.6320±.0067	.6394±.0098	.6360±.0316	**.6508±.0190**	.6259±.0259	.6286±.0097	.6226±.0221	.6378±.0258	.6362±.0018	.6451±.0347
glass_0_1_2_3_vs_4_5_6	.8620±.0173	.8862±.0090	.8712±.0121	.8786±.0131	.8944±.0682	.8620±.0126	.8862±.0057	.8862±.0084	.8862±.0099	.8916±.0171	**.9162±.0154**
new_thyroid1	.9084±.0179	.9396±.0049	.9180±.0086	.9357±.0150	.9391±.0211	.9084±.0158	.9332±.0188	.9413±.0105	.9280±.0165	.9357±.0063	**.9444±.0129**
new_thyroid2	.9263±.0287	.9613±.0082	.9396±.0094	.9491±.0088	.9133±.0549	.9014±.0157	.9692±.0186	.9596±.0036	.9692±.0031	.9396±.0118	**.9713±.0040**
vehicle1	.4768±.0173	.6669±.0016	.6487±.0023	**.6741±.0134**	.6712±.0103	.6662±.0278	.6686±.0070	.6707±.0082	.6721±.0165	.6736±.0099	**.6741±.0062**
vehicle2	.9500±.0016	.9529±.0022	.9525±.0087	.9513±.0003	.9569±.0017	.9466±.0025	**.9650±.0035**	.9609±.0044	.9600±.0008	.9541±.0013	.9593±.0057
vehicle3	.4080±.0067	.6575±.0033	.6510±.0052	.6599±.0066	.6164±.0107	.6616±.0095	.6592±.0108	.6620±.0040	**.6670±.0035**	.6563±.0091	.6640±.0015
pima	.6073±.0033	.6441±.0142	.6464±.0137	.6560±.0070	.6322±.0064	.5829±.0118	.6489±.0011	.6548±.0109	.6516±.0070	.6504±.0092	**.6561±.0049**
ecoli1	.7926±.0116	.7509±.0105	.7417±.0100	.7372±.0039	.7403±.0116	.7693±.0194	.7739±.0149	.8036±.0055	**.8079±.0115**	.7394±.0109	.7755±.0045
ecoli2	.8698±.0186	.8409±.0240	.8345±.0122	.8420±.0062	.8590±.0062	.8600±.0118	.8669±.0020	.8659±.0104	.8728±.0160	.8529±.0117	**.8753±.0224**
ecoli_0_3_4_vs_5	.8148±.0151	.8159±.0038	.7248±.0046	.7629±.0032	.8225±.0054	.7981±.0200	.8248±.0156	.8181±.0056	.8248±.0094	.8248±.0061	**.8295±.0035**
ecoli_0_1_vs_2_3_5	.6659±.0386	.7827±.0067	.8434±.0098	.8434±.0316	.7979±.0190	.8458±.0259	.8389±.0097	.8164±.0231	.8489±.0258	.8379±.0018	**.8592±.0297**
yeast_2_vs_4	.7533±.0359	.7443±.0303	.6762±.0160	.6862±.0226	.7412±.0230	**.7724±.0330**	.7543±.0264	.7214±.0138	.7395±.0214	.6848±.0088	.7592±.0216
yeast_0_3_5_9_vs_7_8	.2663±.0175	**.3863±.0217**	.3495±.0211	.3705±.0257	.3487±.0230	.3663±.0049	.3577±.0513	.3795±.0373	.3745±.0141	.3688±.0266	.3817±.0150
yeast_0_5_6_7_9_vs_4	.1885±.0192	.4808±.0058	.5050±.0124	.4646±.0178	.4836±.0120	.5419±.0180	.5293±.0096	.5497±.0281	**.5527±.0095**	.5104±.0108	.5445±.0258
abalone9_18	.0364±.0184	.3865±.0235	.4074±.0250	.4293±.0251	.4200±.0408	.3964±.0166	.3864±.0176	.3977±.0268	.3601±.0304	**.4405±.0337**	.3986±.0300
haberman	.2761±.0256	.4524±.0148	.4326±.0130	.4360±.0160	.3719±.0279	.4650±.0279	.4692±.0064	.4532±.0124	.4670±.0204	.4709±.0271	**.4868±.0219**
wisconsin	.9580±.0008	.9605±.0022	.9521±.0016	.9519±.0016	.9520±.0016	.9555±.0034	.9580±.0033	.9585±.0002	.9580±.0041	.9519±.0019	**.9625±.0010**
ILPD	.0121±.0057	.5358±.0052	.5404±.0149	.5685±.0142	.3780±.0087	.5670±.0104	.5504±.0153	.5664±.0236	.5491±.0041	.5412±.0117	**.5696±.0079**
seeds_2_vs_1_3	.9644±.0041	.9630±.0008	.9530±.0037	.9590±.0008	.9575±.0008	.9644±.0047	.9575±.0041	.9575±.0008	.9575±.0008	.9590±.0037	**.9670±.0008**
wilt	.7795±.0153	.7783±.0022	.7253±.0026	.7372±.0119	.4742±.0110	.7579±.0219	.8242±.0049	.8559±.0108	**.8589±.0155**	.7334±.0108	.8373±.0069
vowel0	**1.000±.0000**	**1.000±.0000**	**1.000±.0000**	**1.000±.0000**	.9397±.0108	**1.000±.0000**	.9835±.0156	.9755±.0129	.9835±.0179	.9777±.0042	**1.000±.0000**
led7digit_0_2_4_5_6_7_8_9_vs_1	**.7298±.0216**	.5909±.0428	.6766±.0272	.6575±.0415	.6533±.0282	.6886±.0735	.6927±.0323	.6912±.0318	.6748±.0266	.7010±.0512	.6966±.0242

The best result in each data set is labeled in boldface

TABLE 4. G-mean of the comparative algorithms by using SVM classifier.

Data-set	Baseline	ROS	SMOTE	B-SMOTE1	B-SMOTE2	SL-SMOTE	GG-SMOTE	RNG-SMOTE	NCN-SMOTE	SMOTE-IPF	GSMOTE-NFM
glass0	.6743±.0054	.7776±.0223	.7785±.0124	.7715±.0136	.7878±.0180	**.7922±.0125**	.7839±.0034	.7635±.0079	.7627±.0015	.7454±.0043	.7888±.0007
glass1	.6507±.0229	.7022±.0045	.7046±.0060	.7117±.0321	.7161±.0167	.7194±.0204	.7049±.0082	.7034±.0202	**.7238±.0225**	.7099±.0046	.7205±.0272
glass_0_1_2_3_vs_4_5_6	.8989±.0142	.9256±.0041	.9317±.0110	.9351±.0083	.9357±.0865	.9589±.0076	.9456±.0051	.9356±.0095	.9456±.0067	.9471±.0122	**.9643±.0111**
new_thyroid1	.9286±.0143	.9908±.0049	.9863±.0057	.9889±.0053	.9845±.0166	.9286±.0106	.9510±.0113	.9871±.0059	.9379±.0106	.9889±.0062	**.9921±.0056**
new_thyroid2	.9416±.0207	**.9945±.0036**	.9857±.0029	.9887±.0037	.9406±.0482	.9074±.0090	.9824±.0021	.9916±.0065	.9824±.0030	.9857±.0005	**.9945±.0049**
vehicle1	.5882±.0153	.7980±.0022	.7905±.0026	**.8131±.0119**	.8102±.0110	.7938±.0219	.7979±.0049	.8007±.0108	.8044±.0155	.8048±.0108	.8076±.0069
vehicle2	.9584±.0004	.9707±.0013	**.9747±.0045**	.9714±.0011	.9723±.0021	.9541±.0016	.9730±.0019	.9714±.0033	.9681±.0006	.9702±.0020	.9731±.0045
vehicle3	.5245±.0027	.8014±.0032	.7952±.0039	**.8061±.0067**	.7546±.0084	.7992±.0086	.7919±.0102	.7851±.0035	.7900±.0041	.7999±.0088	.7977±.0012
pima	.6861±.0022	.7199±.0146	.7226±.0120	.7210±.0075	.7099±.0072	.7122±.0090	.7154±.0022	.7125±.0108	.7191±.0058	**.7362±.0055**	.7291±.0029
ecoli1	.8586±.0089	.8870±.0082	.8882±.0069	.8781±.0051	.8843±.0023	.8847±.0156	.8900±.0102	.8828±.0043	.8847±.0111	.8808±.0110	**.8921±.0047**
ecoli2	.9103±.0091	.9378±.0138	.9360±.0091	.9379±.0074	.9411±.0067	.9320±.0068	.9376±.0071	.9429±.0051	**.9463±.0088**	.9392±.0090	.9446±.0094
ecoli_0_3_4_vs_5	.8569±.0131	.8687±.0048	.8599±.0056	.8608±.0052	.8609±.0064	.8634±.0100	.8643±.0106	.8620±.0056	.8543±.0094	.8648±.0061	**.8693±.0055**
ecoli_0_1_vs_2_3_5	.6760±.0152	.8497±.0032	.9013±.0056	.9113±.0285	.8747±.0174	.8760±.0133	.8729±.0052	.8546±.0191	.8729±.0187	.8979±.0017	**.9342±.0253**
yeast_2_vs_4	.7912±.0293	.8569±.0195	.8531±.0101	.8652±.0206	.8646±.0210	.8561±.0273	.8602±.0173	.8442±.0115	.8505±.0234	.8616±.0059	**.8790±.0134**
yeast_0_3_5_9_vs_7_8	.3975±.0249	.6692±.0206	.6392±.0291	.6733±.0266	.6871±.0250	.3975±.0160	.6537±.0555	**.6950±.0407**	.6450±.0198	.6591±.0348	.6810±.0093
yeast_0_5_6_7_9_vs_4	.2910±.0237	.7649±.0100	.7825±.0145	.7607±.0064	.7501±.0200	.2491±.0247	.7606±.0038	.7604±.0246	.7599±.0058	.7716±.0047	**.8101±.0310**
abalone9_18	.0632±.0323	.7616±.0224	.7591±.0133	.7685±.0191	.7652±.0479	.7632±.0291	.7875±.0121	.7773±.0248	.7806±.0359	.7680±.0214	**.7920±.0330**
haberman	.4232±.0375	.6039±.0126	.5801±.0087	.5889±.0119	.5128±.0284	.6318±.0377	.6342±.0089	.5762±.0115	.6058±.0162	.6150±.0249	**.6490±.0229**
wisconsin	.9700±.0016	.9739±.0024	.9665±.0023	.9673±.0018	.9675±.0017	.9671±.0034	.9700±.0036	.9721±.0013	.9707±.0036	.9673±.0022	**.9768±.0006**
ILPD	.0365±.0171	.6231±.0061	.6504±.0151	.6825±.0133	.5310±.0075	.6822±.0021	.6759±.0149	.6362±.0186	.6176±.0017	.6639±.0109	**.6834±.0083**
seeds_2_vs_1_3	.9744±.0010	.9799±.0009	.9706±.0019	.9766±.0009	.9708±.0009	.9744±.0030	.9708±.0010	.9708±.0009	.9708±.0009	.9766±.0019	**.9799±.0009**
wilt	.8099±.0041	.9576±.0008	.9634±.0037	.9608±.0008	.5778±.0008	.9576±.0047	.9665±.0041	.9559±.0008	.9544±.0008	**.9726±.0037**	.9669±.0008
vowel0	**1.000±.0000**	**1.000±.0000**	**1.000±.0000**	**1.000±.0000**	.9418±.0098	**1.000±.0000**	.9983±.0014	.9978±.0011	.9983±.0016	.9984±.0003	**1.000±.0000**
led7digit_0_2_4_5_6_7_8_9_vs_1	.8294±.0065	.8298±.0074	.8406±.0087	.8396±.0090	.8278±.0043	.8418±.0420	.8344±.0074	.8443±.0085	.8424±.0108	**.8430±.0043**	.8429±.0122

The best result in each data set is labeled in boldface

statistical difference among a group of results, and Holm post-hoc test is used to examine whether the proposed algorithm is distinctive among a $1 \times N$ comparison [48]–[50]. The post-hoc procedure allows us to know whether a hypothesis of comparison of means can be rejected at a specified level of significance α. The adjusted p-value (APV) is also calculated to denote the lowest level of significance of a hypothesis that results in a rejection. Furthermore, we consider the average

TABLE 5. F-measure of the comparative algorithms by using logistic regression classifier.

Data-set	Baseline	ROS	SMOTE	B-SMOTE1	B-SMOTE2	SL-SMOTE	GG-SMOTE	RNG-SMOTE	NCN-SMOTE	SMOTE-IPF	GSMOTE-NFM
glass0	.5755±.0161	.6158±.0153	.6238±.0123	.6302±.0063	.6148±.0190	.6240±.0032	.6188±.0141	.6338±.0127	.6249±.0085	**.6495±.0102**	.6384±.0044
glass1	.4816±.0206	.5522±.0117	**.5823±.0348**	.5563±.0253	.5573±.0087	.5630±.0060	.5446±.0156	.5517±.0239	.5184±.0352	.5531±.0053	.5786±.0045
glass_0_1_2_3_vs_4_5_6	.8014±.0104	.7879±.0122	.7760±.0330	.8220±.0117	.8050±.0105	.8329±.0345	.8242±.0022	.7939±.0185	.8000±.0156	.8387±.0050	**.8578±.0211**
new_thyroid1	.9483±.0261	.9557±.0100	.8099±.0227	.9483±.0261	**.9818±.0135**	.9568±.0450	.9723±.0324	.9636±.0231	.9723±.0343	.8932±.0094	.9757±.0143
new_thyroid2	.9417±.0068	.9415±.0128	.8764±.0053	.9195±.0086	.9282±.0185	.9034±.5870	**.9464±.0057**	.9064±.0245	**.9464±.0057**	.8824±.0142	.9415±.0018
vehicle1	.5804±.0142	.6558±.0041	.6487±.0110	.6496±.0083	.6507±.0865	.6521±.0076	.6498±.0051	.6482±.0095	.6372±.0067	**.6578±.0122**	.6554±.0111
vehicle2	.9212±.0030	.9184±.0068	.9141±.0062	.9282±.0112	.9261±.0051	.9321±.0025	.9400±.0035	**.9417±.0028**	**.9417±.0046**	.9136±.0056	.9330±.0018
vehicle3	.5429±.0030	.6252±.0134	.6113±.0132	**.6442±.0049**	.6383±.0058	.6300±.0045	.6301±.0145	.5905±.0177	.6249±.0166	.6366±.0061	.6234±.0115
pima	.6383±.0081	.6730±.0063	.6611±.0044	**.6750±.0041**	.6701±.0046	.6700±.0021	.6554±.0014	.6618±.0131	.6660±.0028	.6602±.0111	.6716±.0010
ecoli1	.7589±.0105	.7492±.0132	.7625±.0084	.7585±.0149	.7589±.0091	.7602±.0025	.7678±.0026	**.7756±.0106**	.7517±.0051	.7485±.0187	.7629±.0034
ecoli2	.6744±.0118	.6808±.0071	.6805±.0037	.7006±.0078	.6829±.0085	.6955±.0045	.6568±.0081	.6813±.0114	.6739±.0049	.6840±.0119	**.7048±.0152**
ecoli_0_3_4_vs_5	.6848±.0457	.6095±.0432	.5767±.0295	.5743±.0252	.6262±.0499	.7292±.0533	.6921±.0611	.6952±.0596	.6952±.0548	.5470±.0199	**.7390±.0655**
ecoli_0_1_vs_2_3_5	.6237±.0659	.5832±.0467	.5100±.0188	.5414±.0384	.5685±.0545	.5733±.0244	.6024±.0452	.5831±.0563	.6024±.0417	.5487±.0506	**.6321±.0343**
yeast_2_vs_4	**.7213±.0105**	.6338±.0238	.6295±.0088	.6574±.0149	.6726±.0121	.6500±.0333	.7141±.0143	.6700±.0349	.6851±.0286	.6736±.0241	.6501±.0198
yeast_0_3_5_9_vs_7_8	.3066±.0176	.3517±.0141	.2828±.0270	.3141±.0251	.3095±.0249	.3500±.0221	.3402±.0173	.3317±.0172	.3296±.0205	.3187±.0201	**.3608±.0082**
yeast_0_5_6_7_9_vs_4	**.5061±.0364**	.4105±.0152	.4088±.0074	.3925±.0144	.4164±.0106	.4362±.0235	.4307±.0246	.4659±.0153	.4763±.0100	.3931±.0158	.4018±.0232
abalone9_18	.0291±.0209	.4082±.0346	.3991±.0244	.4254±.0246	.4147±.0395	.4004±.0171	.3857±.0184	.3991±.0279	.3509±.0252	**.4315±.0415**	.3988±.0277
haberman	.2329±.0019	.4543±.0106	.4579±.0134	.4581±.0195	.4663±.0133	.4634±.0034	.4367±.0217	.4562±.0114	.4127±.0103	.4590±.0088	**.4772±.0099**
wisconsin	.9460±.0045	.9479±.0044	.9509±.0006	.9544±.0021	.9467±.0041	.9502±.0076	.9500±.0022	.9519±.0050	.9500±.0019	**.9541±.0033**	.9519±.0005
ILPD	.3111±.0156	.5036±.0056	.5201±.0088	.5661±.0005	.5620±.0061	.5562±.0046	.5195±.0016	.5271±.0030	.5160±.0050	.5641±.0047	**.5673±.0049**
seeds_2_vs_1_3	.9795±.0038	**.9895±.0048**	.9826±.0027	.9752±.0044	.9834±.0054	.9767±.0031	**.9895±.0048**	.9826±.0016	.9826±.0049	.9752±.0052	.9821±.0077
wilt	.7795±.0153	.7783±.0022	.7253±.0026	.7372±.0119	.4742±.0110	.7579±.0219	.8242±.0049	.8559±.0108	**.8589±.0155**	.7334±.0108	.8373±.0069
vowel0	**.8717±.0096**	.8259±.0185	.8025±.0047	.7986±.0118	.8147±.0164	.8023±.0075	.8361±.0041	.8456±.0048	.8503±.0149	.7951±.0101	.8198±.0104
led7digit_0_2_4_5_6_7_8_9_vs_1	.7325±.0296	.5317±.0048	.5013±.0040	.4788±.0135	.5245±.0212	.6796±.0054	.7406±.0173	**.7443±.0344**	.7389±.0096	.5104±.0051	.6853±.0249

The best result in each data set is labeled in boldface

TABLE 6. G-mean of the comparative algorithms by using logistic regression classifier.

Data-set	Baseline	ROS	SMOTE	B-SMOTE1	B-SMOTE2	SL-SMOTE	GG-SMOTE	RNG-SMOTE	NCN-SMOTE	SMOTE-IPF	GSMOTE-NFM
glass0	.6763±.0126	.7079±.0151	.7172±.0096	.7038±.0061	.7077±.0193	.7332±.0043	.7075±.0166	.7226±.0127	.7151±.0080	**.7426±.0109**	.7311±.0053
glass1	.5860±.0157	.6180±.0060	.6356±.0286	.6292±.0171	.6261±.0109	.6354±.0070	.6286±.0118	.6376±.0189	.6116±.0266	.6231±.0027	**.6483±.0062**
glass_0_1_2_3_vs_4_5_6	.8715±.0089	.8662±.0060	.8575±.0210	.8948±.0068	.8765±.0052	.8965±.0357	.8856±.0051	.8586±.0122	.8660±.0088	.8948±.0014	**.9212±.0215**
new_thyroid1	.9616±.0179	.9733±.0123	.9541±.0165	.9785±.0295	.9738±.0174	.9834±.0432	.9795±.0326	.9764±.0105	.9795±.0325	.9738±.0037	**.9880±.0058**
new_thyroid2	.9645±.0029	**.9890±.0073**	.9717±.0041	.9732±.0011	.9763±.0084	.9605±.0049	.9789±.0018	.9554±.0114	.9789±.0018	.9717±.0050	**.9890±.0042**
vehicle1	.6920±.0173	.7908±.0090	.7846±.0121	.7874±.0131	.7845±.0682	.7846±.0126	.7713±.0057	.7718±.0084	.7589±.0099	**.7924±.0171**	.7897±.0154
vehicle2	.9510±.0021	.9540±.0048	.9566±.0036	.9649±.0050	.9607±.0004	.9533±.0046	.9636±.0025	.9627±.0026	.9627±.0032	**.9565±.0034**	.9613±.0019
vehicle3	.6618±.0006	.7695±.0114	.7605±.0115	.7588±.0043	.7607±.0059	.7655±.0056	.7665±.0123	.7338±.0163	.7651±.0140	**.7815±.0060**	.7661±.0099
pima	.7119±.0060	**.7481±.0057**	.7365±.0036	.7397±.0026	.7360±.0030	.7422±.0046	.7332±.0005	.7376±.0110	.7413±.0015	.7376±.0088	.7466±.0011
ecoli1	.8397±.0078	.8667±.0043	.8734±.0070	**.8813±.0068**	.8741±.0078	.8754±.0052	.8632±.0112	.8736±.0004	.8528±.0057	.8697±.0129	.8794±.0036
ecoli2	.7836±.0089	**.8981±.0079**	.8906±.0036	.8980±.0072	.8836±.0037	.8947±.0142	.8610±.0108	.8831±.0106	.8667±.0101	.8867±.0085	.8907±.0144
ecoli_0_3_4_vs_5	.7991±.0716	.8082±.0686	.8584±.0779	.8454±.0724	.8130±.0733	.8853±.0621	.8830±.0921	.8715±.0879	.8715±.0790	.8385±.0783	**.8919±.0794**
ecoli_0_1_vs_2_3_5	.7955±.0625	.8063±.0606	.7918±.0582	.7989±.0751	.8048±.0740	.8155±.0655	.8112±.0731	.8069±.0766	.8112±.0723	.8002±.0734	**.8359±.0571**
yeast_2_vs_4	.7942±.0069	.8694±.0137	.8814±.0058	.8875±.0127	.8771±.0042	.8810±.0227	.8801±.0101	.8478±.0163	.8442±.0175	**.9115±.0189**	.8852±.0159
yeast_0_3_5_9_vs_7_8	.4361±.0304	.7225±.0158	.6609±.0316	.6801±.0206	.6739±.0235	.7144±.0032	.6869±.0157	.6843±.0177	.6681±.0204	.7024±.0121	**.7179±.0098**
yeast_0_5_6_7_9_vs_4	.6066±.0286	.7644±.0148	.7290±.0136	.7372±.0127	**.7675±.0116**	.7532±.0231	.7274±.0223	.7548±.0199	.7548±.0081	.7451±.0126	.7408±.0265
abalone9_18	.0818±.0293	.7911±.0224	.7591±.0133	.7672±.0181	.7677±.0514	.7682±.0248	.7275±.0121	.7073±.0248	.7106±.0359	.7784±.0292	**.7981±.0384**
haberman	.3443±.0170	.6013±.0093	.6055±.0113	.6028±.0183	.6126±.0104	**.6259±.0033**	.5947±.0214	.6143±.0118	.5756±.0122	.6035±.0113	.6240±.0093
wisconsin	.9570±.0036	.9600±.0037	.9645±.0021	**.9689±.0018**	.9605±.0031	.9642±.0542	.9609±.0020	.9638±.0050	.9609±.0028	.9670±.0029	.9688±.0012
ILPD	.4554±.0123	.6488±.0060	.6532±.0073	.6631±.0011	.6778±.0054	.6752±.0081	.6547±.0021	.6594±.0061	.6522±.0017	.6811±.0055	**.6897±.0046**
seeds_2_vs_1_3	.9869±.0056	.9969±.0059	.9893±.0031	.9599±.0032	.9910±.0048	.9900±.0059	.9969±.0059	.9933±.0043	.9933±.0059	.9899±.0034	**.9975±.0075**
wilt	.8154±.0059	.9676±.0008	**.9734±.0037**	.9708±.0008	.5782±.0172	.7876±.0047	.9465±.0041	.9566±.0016	.9591±.0100	.9719±.0077	.9669±.0008
vowel0	.9185±.0034	.9742±.0124	**.9752±.0026**	.9746±.0058	.9629±.0151	.9511±.0034	.9457±.0034	.9406±.0035	.9521±.0071	.9741±.0060	.9644±.0066
led7digit_0_2_4_5_6_7_8_9_vs_1	.8642±.0068	.8731±.0025	.8678±.0030	.8601±.0068	.8511±.0124	.8456±.0153	.8902±.0031	**.8903±.0020**	.8902±.0092	.8696±.0016	.8820±.0113

The best result in each data set is labeled in boldface

rankings of the algorithms in order to measure how good an algorithm is with respect to its opponent. The ranking can be calculated by assigning a position to each algorithm depending on its performance on each data set. The algorithm that achieves the best performance on a specific data set will be given rank 1 (value 1), then the algorithm with the second best result will be assigned rank 2, and so forth. This task is carried out over all data sets and finally an average

TABLE 7. F-measure of the comparative algorithms by using C4.5 decision tree.

Data-set	Baseline	ROS	SMOTE	B-SMOTE1	B-SMOTE2	SL-SMOTE	GG-SMOTE	RNG-SMOTE	NCN-SMOTE	SMOTE-IPF	GSMOTE-NFM
glass0	.6587±.0291	.6621±.0191	.6632±.0203	.6829±.0274	.6818±.0474	.6810±.0453	.6924±.0131	.6665±.0204	.6760±.0306	**.7004±.0136**	.6859±.0481
glass1	.6144±.0218	.6428±.0193	.6789±.0262	.6531±.0178	.6419±.0129	.6534±.0123	.6512±.0195	.6524±.0201	.6478±.0086	.6454±.0217	**.6862±.0273**
glass_0_1_2_3_vs_4_5_6	.8227±.0234	.8174±.0260	.8348±.0242	.8664±.0509	.8091±.0051	.8064±.0170	.8677±.0191	**.8898±.0115**	.8549±.0170	.8696±.0150	.8741±.0205
new_thyroid1	.8272±.0101	.8801±.0158	.8129±.0155	.8257±.0149	.8887±.0108	.8733±.0378	.8848±.0207	.8896±.0155	.8884±.0331	.8907±.0208	**.8911±.0320**
new_thyroid2	.8247±.0170	.8882±.0110	.8347±.0258	.8880±.0124	**.9134±.0197**	.8730±.0202	.8448±.0308	.8549±.0283	.8478±.0307	.8468±.0261	.8946±.0092
vehicle1	.5266±.0198	.4861±.0166	.5370±.0234	.4999±.0165	.5031±.0078	.5036±.0199	.5126±.0139	.5070±.0185	.5226±.0066	.4857±.0168	**.5720±.0191**
vehicle2	.9500±.0016	**.9729±.0022**	.9525±.0087	.9613±.0003	.9569±.0017	.9466±.0025	.9050±.0035	.9509±.0044	.9500±.0008	.9541±.0013	.9603±.0057
vehicle3	.5200±.0004	.4910±.0013	.5257±.0045	.5505±.0011	.5441±.0021	.4899±.0016	.5233±.0019	.5422±.0033	.5400±.0006	**.5516±.0020**	.5452±.0045
pima	.6073±.0003	.6441±.0145	.6464±.0112	.6560±.0071	.6322±.0061	.6229±.0050	.6389±.0018	.6248±.0083	.6016±.0043	.6504±.0049	**.6561±.0031**
ecoli1	.7433±.0162	.7807±.0206	.7483±.0189	.7136±.0055	.7825±.0094	.7157±.0193	.7307±.0141	.7627±.0104	.7312±.0204	.7075±.0137	**.7995±.0100**
ecoli2	.6755±.0537	.6912±.0395	.6787±.0290	.6953±.0299	.6627±.0591	.6429±.0489	.7093±.0070	.6569±.0179	**.7192±.0173**	.6762±.0184	.6834±.0370
ecoli_0_3_4_vs_5	.7148±.0151	.7159±.0038	.6248±.0046	.6629±.0032	**.7425±.0054**	.6981±.0200	.6748±.0156	.6481±.0056	.6748±.0094	.7248±.0061	.7295±.0035
ecoli_0_1_vs_2_3_5	.6495±.0152	.6944±.0032	.6616±.0056	.6593±.0285	.6724±.0174	**.7019±.0133**	.6865±.0052	.5480±.0191	.6026±.0187	.6742±.0017	.6943±.0253
yeast_2_vs_4	.6437±.0112	.6344±.0278	.6820±.0462	.6436±.0135	.6866±.0276	.6053±.0211	.7097±.0191	.6045±.0233	.6649±.0165	.6584±.0259	**.7212±.0427**
yeast_0_3_5_9_vs_7_8	.2695±.0715	.2297±.0383	.3217±.0257	.2943±.0178	**.3555±.0342**	.2581±.0446	.3450±.0234	.2168±.0338	.2868±.0136	.3513±.0331	.3276±.0554
yeast_0_5_6_7_9_vs_4	.3190±.0058	.4345±.0431	.4181±.0274	.4452±.0349	.4869±.0184	.3850±.0050	.4378±.0404	.3964±.0214	.3893±.0345	**.4875±.0249**	.4536±.0112
abalone9_18	.3210±.0072	.2189±.0045	**.3432±.0010**	.3249±.0193	.2794±.0290	.2737±.0694	.2729±.0828	.1930±.0337	.2366±.0375	.2245±.0398	.2871±.0287
haberman	.3460±.0086	.3768±.0343	.3613±.0160	.3649±.0190	.3509±.0392	.2546±.0059	.3361±.0164	.3619±.0059	.3417±.0019	.3480±.0126	**.4154±.0198**
wisconsin	.9262±.0076	.9300±.0039	.9280±.0065	.9239±.0077	**.9405±.0065**	.9384±.0109	.9338±.0099	.9409±.0052	.9378±.0135	.9301±.0057	.9328±.0076
ILPD	.4303±.0265	.4426±.0364	.4651±.0183	.4126±.0145	.4249±.0056	.3908±.0117	.4130±.0175	.4208±.0322	**.4943±.0442**	.4733±.0246	.4524±.0227
seeds_2_vs_1_3	**.9683±.0045**	.9404±.0036	.9461±.0041	.9630±.0116	.9586±.0074	.9596±.0026	**.9683±.0084**	**.9683±.0084**	.9700±.0074	.9522±.0071	.9672±.0133
wilt	.7858±.0042	.7841±.0029	.6382±.0064	.6556±.0073	.7669±.0071	.7492±.0031	.7466±.0051	.7065±.0051	.6245±.0052	.6254±.0037	**.7867±.0062**
vowel0	.8960±.0167	.8878±.0324	.8893±.0293	.8916±.0279	.9153±.0263	**.9397±.0280**	.8955±.0265	.8822±.0438	.8947±.0325	.8986±.0425	.9071±.0052
led7digit_0_2_4_5_6_7_8_9_vs_1	.5946±.0177	.5454±.0334	.6591±.0159	**.7022±.0149**	.5311±.0202	.6647±.0176	.6246±.0254	.6054±.0497	.6345±.0445	.6472±.0333	.6728±.0534

The best result in each data set is labeled in boldface

TABLE 8. G-mean of the comparative algorithms by using C4.5 decision tree.

Data-set	Baseline	ROS	SMOTE	B-SMOTE1	B-SMOTE2	SL-SMOTE	GG-SMOTE	RNG-SMOTE	NCN-SMOTE	SMOTE-IPF	GSMOTE-NFM
glass0	.7443±.0157	.7474±.0154	.7552±.0254	.7516±.0174	.7643±.0290	.7429±.0330	.7744±.0196	.7585±.0097	.7548±.0277	**.7865±.0163**	.7615±.0462
glass1	.6902±.0198	.7187±.0166	.7154±.0234	.7271±.0165	.7213±.0078	.7311±.0199	.7335±.0139	.7237±.0185	.7380±.0066	.7215±.0168	**.7457±.0191**
glass_0_1_2_3_vs_4_5_6	.8769±.0189	.8894±.0175	.9145±.0100	.8991±.0471	.8817±.0036	.8526±.0159	.9216±.0130	.9384±.0051	.9022±.0084	.9385±.0133	**.9414±.0102**
new_thyroid1	.9032±.0087	.9058±.0066	.9055±.0077	.9102±.0085	.9294±.0079	.9224±.0224	.9169±.0084	.9232±.0217	.9326±.0137	**.9532±.0126**	.9379±.0093
new_thyroid2	.8806±.0077	.9170±.0069	.9343±.0204	.9509±.0028	.9444±.0150	.9207±.0118	.9392±.0169	.8889±.0137	.9180±.0190	.9408±.0073	**.9512±.0095**
vehicle1	.6593±.0218	.6354±.0193	.6851±.0262	.6480±.0178	.6508±.0129	.6359±.0123	.6516±.0195	.6548±.0201	.6645±.0086	.6384±.0217	**.7002±.0273**
vehicle2	.9584±.0004	**.9807±.0013**	.9747±.0045	.9614±.0011	.9723±.0021	.9541±.0016	.9730±.0019	.9714±.0033	.9681±.0006	.9742±.0020	.9751±.0045
vehicle3	.6564±.0016	.6385±.0022	.6786±.0087	**.6952±.0003**	.6933±.0017	.6324±.0025	.6333±.0035	.6446±.0044	.6355±.0008	.6787±.0013	.6876±.0057
pima	.6861±.0033	.7199±.0142	.7026±.0137	.7210±.0070	.7099±.0064	.7216±.0118	.7154±.0011	.7025±.0109	.7091±.0070	.7262±.0092	**.7291±.0049**
ecoli1	.8178±.0193	.8603±.0181	.8333±.0178	.8310±.0059	.8745±.0048	.7888±.0111	.8264±.0164	.8502±.0078	.8183±.0118	.8319±.0091	**.8888±.0064**
ecoli2	.7652±.0505	.8243±.0163	.8300±.0157	.8385±.0207	.8571±.0212	.7406±.0529	.8412±.0028	.8128±.0151	**.8621±.0103**	.8438±.0138	.8434±.0145
ecoli_0_3_4_vs_5	.7569±.0131	.7687±.0048	.7799±.0056	.7808±.0052	**.7809±.0064**	.7434±.0100	.7543±.0106	.7520±.0056	.7543±.0094	.7748±.0061	.7693±.0055
ecoli_0_1_vs_2_3_5	.7871±.0386	.8396±.0067	.8377±.0098	.8066±.0316	.7975±.0190	.7154±.0259	**.8417±.0097**	.6779±.0231	.7958±.0258	.8137±.0018	.8310±.0297
yeast_2_vs_4	.7447±.0310	.7712±.0073	.8397±.0166	.8566±.0063	.8296±.0119	.7023±.0223	.8409±.0256	.7602±.0138	.8294±.0264	.8367±.0301	**.8570±.0138**
yeast_0_3_5_9_vs_7_8	.4883±.0760	.4634±.0392	.6326±.0420	.6089±.0247	.6454±.0538	.4468±.0554	.6102±.0186	.4862±.0316	.5497±.0376	.6367±.0429	**.6770±.0667**
yeast_0_5_6_7_9_vs_4	.5474±.0167	.6555±.0324	.7158±.0293	.7108±.0279	.7242±.0263	.5637±.0280	.7037±.0265	.6915±.0438	.6795±.0325	.7073±.0425	**.7250±.0052**
abalone9_18	.4789±.0329	.4326±.0345	.5658±.0316	.5835±.0171	.5513±.0291	.4277±.0877	.5795±.1372	.4773±.0604	.5400±.0564	.4911±.0648	**.5976±.0295**
haberman	.5076±.0118	.5405±.0289	.5249±.0161	.5312±.0144	.5103±.0310	.4148±.0021	.5007±.0223	.5196±.0025	.5062±.0027	.5211±.0108	**.5731±.0124**
wisconsin	.9022±.0065	.9448±.0048	.9488±.0038	.9430±.0076	**.9563±.0067**	.9495±.0074	.9489±.0091	.9577±.0033	.9512±.0111	.9499±.0038	.9529±.0017
ILPD	.5763±.0226	.5862±.0301	.6055±.0153	.5642±.0123	.5734±.0059	.5267±.0122	.5623±.0143	.5685±.0269	**.6314±.0355**	.6132±.0209	.5955±.0185
seeds_2_vs_1_3	.9748±.0042	.9551±.0029	.9644±.0064	.9734±.0073	.9691±.0071	.9655±.0031	.9748±.0051	.9748±.0051	.9744±.0052	.9627±.0037	**.9768±.0062**
wilt	.8796±.0045	.8681±.0036	.8891±.0041	.8998±.0116	**.9272±.0074**	.8467±.0026	.8433±.0084	.9025±.0084	.9245±.0074	.9088±.0071	.8979±.0133
vowel0	.9495±.0058	.9278±.0431	.9578±.0274	.9822±.0349	.9857±.0184	**.9651±.0050**	.9566±.0404	.9546±.0214	.9612±.0345	.9640±.0249	.9645±.0112
led7digit_0_2_4_5_6_7_8_9_vs_1	.7486±.0055	.7354±.0263	.8324±.0097	.8069±.0218	.7479±.0205	.7953±.0230	.7861±.0095	.8034±.0165	.7943±.0048	.8186±.0237	**.8334±.0492**

The best result in each data set is labeled in boldface

ranking is calculated. The statistical results are presented in Table 9-Table 11.

The statistical results present that the proposed GSMOTE-NFM algorithm has acquired the lowest average ranking values in terms of both F-measure and G-mean metrics with all classifiers, indicating it is best among all sampling algorithms. In addition, we observe that in terms of SVM and Logistic Regression classifiers, the APVs of all other algorithms are lower than a standard level of significance of $\alpha = 0.05$. That means these algorithms are

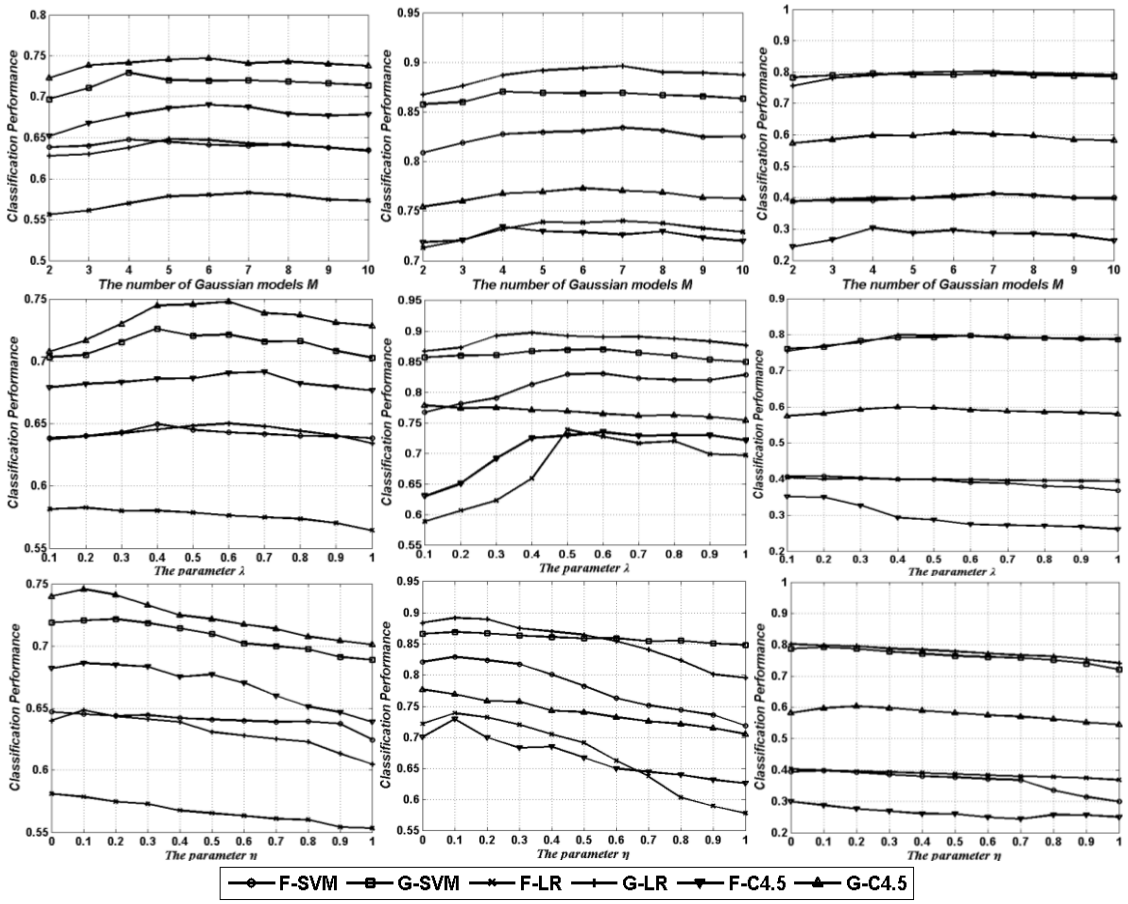

FIGURE 5. The variation trends of the classification performance associating with the variation of parameters, where from top to bottom, each row respectively corresponds to a specific parameter, i.e., M, λ and η, and from left to right, each column corresponds to a specific data set, i.e., glass1, ecoli_0_3_4_vs_5, abalone9_18, as well the curve in each sub-graph indicate F-measure or G-mean metric associating with a specific classifier, SVM, LR(Logistic Regression) or C4.5 decision tree, respectively.

significantly different from the proposed GSMOTE-NFM algorithm. As for C4.5 decision tree classifier, GSMOTE-NFM is significantly better than all comparative algorithms in terms of G-mean, but on F-measure, GSMOTE-NFM has no significant differences from B-SMOTE2, SL-SMOTE, GG-SMOTE, RNG-SMOTE and NCN-SMOTE. However, GSMOTE-NFM clearly has a smaller ranking_F value than those five algorithms, indicating its superiority. Therefore, we can safely recommend the proposed GSMOTE-NFM algorithm as an efficient choice for learning from imbalanced data.

E. PARAMETER DISCUSSION

There are three key parameters in our proposed algorithm, the number of Gaussian models in GMM M, the parameter λ, and the parameter η. Next, we will investigate the influence of the variation of these three parameters. Specifically, M changes from 2 to 10 with an increment 1, λ varies from 0.1 to 1.0 with an increment 0.1, and η changes from 0 to 1 with an increment 0.1. When a parameter varies, two other ones use the default values. The variation curves on three representative data sets, i.e., glass1, ecoli_0_3_4_vs_5, abalone9_18 which respectively correspond to the low, medium and high imbalance ratio, are presented in Fig.5.

Firstly, we investigate the impact of the parameter M. For the curves of the 1st row sub-graphs in Fig.5, although there exist some random fluctuations, a common trend can be still observed, i.e., with the increase of the number of Gaussian models M, the classification performance including F-measure and G-mean first significantly improve, and then they generally have the distinct declines. It is not difficult to understand the variation tendency. A small M tends to make GMM model underfit, while a large M is apt to make GMM model overfit [42]. No matter underfit or overfit, an inaccurate probability density estimation result will be provided to further misguide the noise filtering and group division.

As for the parameter λ, we know it relates with the noisy instances filtering, as well the division between boundary group and safety group. A similar variation trend as that acquired by the parameter M has also been observed for the parameter λ. Obviously, a small λ value will hinder the

TABLE 9. Under SVM classifier, Average Friedman rankings and APVs of various algorithms using Holm's procedure in F-measure and G-mean, where ranking_F denotes average rankings on F-measure, ranking_G denotes average rankings on G-mean measure, APV_F denotes the adjusted *p*-value using Holm's procedure in F-measure and APV_G denotes adjusted *p*-value using Holm's procedure in G-mean, respectively.

Algorithm	ranking_F	APV_F	ranking_G	APV_G
GSMOTE-NFM	1.958	--	1.563	--
Baseline	8.146	$\mathbf{1.029\times10^{-9}}$	9.896	$\mathbf{3.207\times10^{-17}}$
ROS	6.417	$\mathbf{1.929\times10^{-5}}$	5.604	$\mathbf{7.284\times10^{-5}}$
SMOTE	7.771	$\mathbf{1.144\times10^{-8}}$	6.500	$\mathbf{1.505\times10^{-6}}$
B-SMOTE1	6.375	$\mathbf{1.984\times10^{-5}}$	4.917	$\mathbf{9.190\times10^{-4}}$
B-SMOTE2	7.354	$\mathbf{1.394\times10^{-7}}$	7.021	$\mathbf{1.072\times10^{-7}}$
SL-SMOTE	6.667	$\mathbf{6.130\times10^{-6}}$	6.646	$\mathbf{7.700\times10^{-7}}$
GG-SMOTE	5.521	$\mathbf{5.955\times10^{-4}}$	5.708	$\mathbf{5.960\times10^{-5}}$
RNG-SMOTE	5.208	**0.001**	6.479	$\mathbf{1.505\times10^{-6}}$
NCN-SMOTE	4.458	**0.009**	6.792	$\mathbf{3.773\times10^{-7}}$
SMOTE-IPF	6.125	$\mathbf{5.398\times10^{-5}}$	4.875	$\mathbf{9.190\times10^{-4}}$

For a standard level of significance of α=0.05, the results which have significant difference compared with the proposed algorithm have been highlighted in boldface.

TABLE 10. Under logistic regression classifier, average Friedman rankings and APVs of various algorithms using Holm's procedure in F-measure and G-mean, where ranking_F denotes average rankings on F-measure, ranking_G denotes average rankings on G-mean measure, APV_F denotes the adjusted *p*-value using Holm's procedure in F-measure and APV_G denotes adjusted *p*-value using Holm's procedure in G-mean, respectively.

Algorithm	ranking_F	APV_F	ranking_G	APV_G
GSMOTE-NFM	3.563	--	2.729	--
Baseline	7.313	$\mathbf{8.078\times10^{-4}}$	10.417	$\mathbf{9.801\times10^{-15}}$
ROS	6.500	**0.015**	5.292	**0.030**
SMOTE	8.167	$\mathbf{1.518\times10^{-5}}$	6.625	$\mathbf{3.776\times10^{-4}}$
B-SMOTE1	6.250	**0.030**	5.208	**0.030**
B-SMOTE2	5.792	0.100	6.625	$\mathbf{3.776\times10^{-4}}$
SL-SMOTE	5.500	0.129	4.917	**0.035**
GG-SMOTE	5.313	0.135	6.271	**0.001**
RNG-SMOTE	5.250	0.135	6.188	**0.002**
NCN-SMOTE	5.750	0.100	6.729	$\mathbf{2.648\times10^{-4}}$
SMOTE-IPF	6.604	**0.012**	5.000	**0.035**

For a standard level of significance of α=0.05, the results which have significant difference compared with the proposed algorithm have been highlighted in boldface.

noise instance discovery, and meanwhile estimate almost all minority-class instances as the boundary ones, further degrade the model to be a 3-nearest neighbors SMOTE. While a large λ value may accidentally delete some instances as noise, and meanwhile narrow the boundary group, which will also degrade the quality of the sampling.

The variation curves associating with the parameter η are quite different from that related with two other parameters. In general, a small η value can produce the best classification performance as in most data sets, the ratio of the outliers is quite small. With the increase of the parameter η, the overfitting phenomenon may emerge to further degrade the quality of the model.

TABLE 11. Under C4.5 Decision Tree classifier, Average Friedman rankings and APVs of various algorithms using Holm's procedure in F-measure and G-mean, where ranking_F denotes average rankings on F-measure, ranking_G denotes average rankings on G-mean measure, APV_F denotes the adjusted *p*-value using Holm's procedure in F-measure and APV_G denotes adjusted *p*-value using Holm's procedure in G-mean, respectively.

Algorithm	ranking_F	APV_F	ranking_G	APV_G
GSMOTE-NFM	2.667	--	2.250	--
Baseline	7.458	$\mathbf{5.594\times10^{-6}}$	8.830	$\mathbf{5.538\times10^{-11}}$
ROS	6.708	$\mathbf{1.700\times10^{-4}}$	7.625	$\mathbf{1.582\times10^{-7}}$
SMOTE	6.375	$\mathbf{5.370\times10^{-4}}$	5.375	**0.004**
B-SMOTE1	5.792	**0.003**	5.250	**0.005**
B-SMOTE2	5.042	**0.013**	4.625	**0.026**
SL-SMOTE	7.208	$\mathbf{1.889\times10^{-5}}$	8.958	$\mathbf{2.441\times10^{-11}}$
GG-SMOTE	5.896	**0.003**	6.021	$\mathbf{4.919\times10^{-4}}$
RNG-SMOTE	6.958	$\mathbf{5.902\times10^{-5}}$	6.750	$\mathbf{1.820\times10^{-5}}$
NCN-SMOTE	6.479	$\mathbf{4.100\times10^{-4}}$	5.938	$\mathbf{5.871\times10^{-4}}$
SMOTE-IPF	5.417	**0.008**	4.375	**0.027**

For a standard level of significance of α=0.05, the results which have significant difference compared with the proposed algorithm have been highlighted in boldface.

V. CONCLUSION

In this paper, a modified SMOTE algorithm is proposed to address class imbalance learning problem. Specifically, Gaussian mixture model is adopted to accurately estimate the probability density of each training instance, further discovery and filter the noisy instances, as well divide the instances into different groups according to their characteristics of distributions to implement individual sampling. In comparison with some other SMOTE-based algorithms which consider the noise filtering or individual sampling, the proposed GSMOTE-NFM algorithm generally has a better adaptivity and robustness. The experimental results have indicated that the proposed algorithm performs significantly better than those traditional oversampling algorithms and SMOTE-based modification algorithms in statistics.

However, GSMOTE-NFM algorithm has two inherent drawbacks. The first one is that its time-complexity is generally higher than some other oversampling algorithms as the procedure of constructing a GMM is time-consuming. The other defect lies in that it is unsuitable for highly imbalanced classification tasks as the extremely sparse minority class instances can not guarantee the accuracy of GMM model.

In future work, we wish to study the alternative strategy for GMM to accelerate the procedure of probability density estimation. In addition, an improved solution of GSMOTE-NFM for classifying multi-class imbalanced data will be investigated, too.

REFERENCES

[1] H. Kaur, H. S. Pannu, and A. K. Malhi, "A systematic review on imbalanced data challenges in machine learning: Applications and solutions," *ACM Comput. Surv.*, vol. 52, no. 4, 2019, Art. no. 79.

[2] G. Haixiang, L. Yijing, J. Shang, G. Mingyun, H. Yuanyue, and G. Bing, "Learning from class-imbalanced data: Review of methods and applications," *Expert Syst. Appl.*, vol. 73, pp. 220–239, May 2017.

[3] N. Japkowicz, "Learning from imbalanced data sets: A comparison of various strategies," in *Proc. Amer. Assoc. Artif. Intell. (AAAI) Workshop*, 2000, pp. 10–15.

[4] N. V. Chawla, N. Japkowicz, and A. Kolcz, "Workshop learning from imbalanced data sets II," in *Proc. Int. Conf. Mach. Learn.*, 2003, pp. 1–3.
[5] N. V. Chawla, N. Japkowicz, and Z. H. Zhou, "Workshop on data mining when classes are imbalanced and errors have costs," in *Proc. 13th Pacific-Asia Knowl. Discovery Data Mining Conf.*, 2009, pp. 1–2.
[6] S. Wang, L. L. Minku, N. Chawla, and X. Yao, in *Proc. IJCAI Workshop Learn. Presence Class Imbalance Concept Drift (LPCICD)*, 2017, pp. 1–3.
[7] N. V. Chawla, N. Japkowicz, and A. Kolcz, "Editorial: Special issue on learning from imbalanced data sets," *ACM SIGKDD Explor. Newslett.*, vol. 6, no. 1, pp. 1–6, Jun. 2004.
[8] Q. Yang and X. Wu, "10 Challenging problems in data mining research," *Int. J. Inf. Technol. Decis. Making*, vol. 5, no. 4, pp. 597–604, 2006.
[9] W. Wei, J. Li, L. Cao, Y. Ou, and J. Chen, "Effective detection of sophisticated Online banking fraud on extremely imbalanced data," *World Wide Web*, vol. 16, no. 4, pp. 449–475, 2013.
[10] A. Zakaryazad and E. Duman, "A profit-driven artificial neural network (ANN) with applications to fraud detection and direct marketing," *Neurocomputing*, vol. 175, pp. 121–131, Jan. 2016.
[11] V. Engen, J. Vincent, and K. Phalp, "Enhancing network based intrusion detection for imbalanced data," *Int. J. Knowl.-Based Intell. Eng. Syst.*, vol. 12, nos. 5–6, pp. 357–367, 2008.
[12] R. Abdulhammed, M. Faezipour, A. Abuzneid, and A. AbuMallouh, "Deep and machine learning approaches for anomaly-based intrusion detection of imbalanced network traffic," *IEEE Sensors Lett.*, vol. 3, no. 1, Jan. 2019, Art. no. 7101404.
[13] M. Zhu, J. Xia, X. Jin, M. Yan, G. Cai, J. Yan, and G. Ning, "Class weights random forest algorithm for processing class imbalanced medical data," *IEEE Access*, vol. 6, pp. 4641–4652, 2018.
[14] H. Han, M. Huang, Y. Zhang, and J. Liu, "Decision support system for medical diagnosis utilizing imbalanced clinical data," *Appl. Sci.*, vol. 8, no. 9, 2018, Art. no. 1597.
[15] S. Wang and X. Yao, "Using class imbalance learning for software defect prediction," *IEEE Trans. Rel.*, vol. 62, no. 2, pp. 434–443, Jun. 2013.
[16] D. Rodriguez, I. Herraiz, R. Harrison, J. Dolado, and J. C. Riquelme, "Preliminary comparison of techniques for dealing with imbalance in software defect prediction," in *Proc. 18th Int. Conf. Eval. Assessment Softw. Eng.*, 2014, Art. no. 43.
[17] M. Kirlidog and C. Asuk, "A fraud detection approach with data mining in health insurance," *Procedia-Soc. Behav. Sci.*, vol. 62, pp. 989–994, Oct. 2012.
[18] R. Kumar, A. Srivastava, B. Kumari, and M. Kumar, "Prediction of β-lactamase and its class by Chou's pseudo-amino acid composition and support vector machine," *J. Theor. Biol.*, vol. 365, pp. 96–103, Jan. 2015.
[19] L. Sun, H. Liu, L. Zhang, and J. Meng, "lncRScan-SVM: A tool for predicting long non-coding RNAs using support vector machine," *PLoS One*, vol. 10, no. 10, 2015, Art. no. e0139654.
[20] C.-M. Vong, W.-F. Ip, C.-C. Chiu, and P.-K. Wong, "Imbalanced learning for air pollution by meta-cognitive online sequential extreme learning machine," *Cognit. Comput.*, vol. 7, no. 3, pp. 381–391, 2015.
[21] A. Azaria, A. Richardson, S. Kraus, and V. Subrahmanian, "Behavioral analysis of insider threat: A survey and bootstrapped prediction in imbalanced data," *IEEE Trans. Comput. Social Syst.*, vol. 1, no. 2, pp. 135–155, Jun. 2014.
[22] R. Batuwita and V. Palade, "Efficient resampling methods for training support vector machines with imbalanced datasets," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2010, pp. 1–8.
[23] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, no. 1, pp. 321–357, 2002.
[24] H. Yu, J. Ni, and J. Zhao, "ACOSampling: An ant colony optimization-based undersampling method for classifying imbalanced DNA microarray data," *Neurocomputing*, vol. 101, no. 3, pp. 309–318, 2013.
[25] Y. Sun, M. S. Kamel, A. K. C. Wong, and Y. Wang, "Cost-sensitive boosting for classification of imbalanced data," *Pattern Recognit.*, vol. 40, no. 12, pp. 3358–3378, 2007.
[26] C. L. Castro and A. P. Braga, "Novel cost-sensitive approach to improve the multilayer perceptron performance on imbalanced data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 24, no. 6, pp. 888–899, Jun. 2013.
[27] S. Datta and S. Das, "Near-Bayesian support vector machines for imbalanced data classification with equal or unequal misclassification costs," *Neural Netw.*, vol. 70, pp. 39–52, Oct. 2015.
[28] H. Yu, C. Sun, X. Yang, S. Zheng, Q. Wang, and X. Xi, "LW-ELM: A fast and flexible cost-sensitive learning framework for classifying imbalanced data," *IEEE Access*, vol. 6, pp. 28488–28500, 2018.
[29] Z.-H. Zhou and X.-Y. Liu, "Training cost-sensitive neural networks with methods addressing the class imbalance problem," *IEEE Trans. Knowl. Data Eng.*, vol. 18, no. 1, pp. 63–77, Jan. 2006.
[30] H. Yu, C. Mu, C. Sun, W. Yang, X. Yang, and X. Zuo, "Support vector machine-based optimized decision threshold adjustment strategy for classifying imbalanced data," *Knowl.-Based Syst.*, vol. 76, pp. 67–78, Mar. 2015.
[31] H. Yu, C. Sun, X. Yang, W. Yang, J. Shen, and Y. Qi, "ODOC-ELM: Optimal decision outputs compensation-based extreme learning machine for classifying imbalanced data," *Knowl.-Based Syst.*, vol. 92, pp. 55–70, Jan. 2016.
[32] M. Galar, A. Fernandez, E. Barrenechea, H. Bustince, and F. Herrera, "A review on ensembles for the class imbalance problem: Bagging-, boosting-, and hybrid-based approaches," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 42, no. 4, pp. 463–484, Jul. 2012.
[33] S. Wang, L. Minku, and X. Yao, "Resampling-based ensemble methods for online class imbalance learning," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 5, pp. 1356–1368, May 2015.
[34] C. Seiffert, T. M. Khoshgoftaar, J. Van Hulse, and A. Napolitano, "RUSBoost: A hybrid approach to alleviating class imbalance," *IEEE Trans. Syst., Man, Cybern. A, Syst., Humans*, vol. 40, no. 1, pp. 185–197, Jan. 2010.
[35] P. Lim, C. K. Goh, and K. C. Tan, "Evolutionary cluster-based synthetic oversampling ensemble (ECO-ensemble) for imbalance learning," *IEEE Trans. Cybern.*, vol. 47, no. 9, pp. 2850–2861, Sep. 2017.
[36] G. He, H. Han, and W. Wang, "An over-sampling expert system for learing from imbalanced data sets," in *Proc. 2nd Int. Conf. Neural Netw. Brain*, Oct. 2005, pp. 537–541.
[37] J. van Hulse, T. M. Khoshgoftaar, and A. Napolitano, "Experimental perspectives on learning from imbalanced data," in *Proc. 24th Int. Conf. Mach. Learn.*, 2007, pp. 935–942.
[38] H. Han, W.-Y. Wang, and B.-H. Mao, "Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning," in *Proc. Int. Conf. Intell. Comput.*, 2005, pp. 878–887.
[39] C. Bunkhumpornpat, K. Sinapiromsaran, and C. Lursinsap, "Safe-level-SMOTE: Safe-level-synthetic minority over-sampling technique for handling the class imbalanced problem," in *Proc. Pacific–Asia Conf. Knowl. Discovery Data Mining*, 2009, pp. 475–482.
[40] J. A. Sáez, J. Luengo, J. Stefanowski, and F. Herrera, "SMOTE-IPF: Addressing the noisy and borderline examples problem in imbalanced classification by a re-sampling method with filtering," *Inf. Sci.*, vol. 291, pp. 184–203, Jan. 2015.
[41] V. García, J. S. Sánchez, R. Martìn-Félez, and R. A. Mollineda, "Surrounding neighborhood-based SMOTE for learning from imbalanced data sets," *Prog. Artif. Intell.*, vol. 1, no. 4, pp. 347–362, 2012.
[42] T. Huang, H. Peng, and K. Zhang, "Model selection for Gaussian mixture models," *Statistica Sinica*, vol. 27, no. 1, pp. 147–169, Jan. 2017.
[43] C. Blake, E. Keogh, and C. J. Merz, "UCI repository of machine learning databases," Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, Tech. Rep. 213, 1998. [Online]. Available: http://www.ics.uci.edu/mlearn/MLReposi-tory.html
[44] I. Triguero, S. González, J. M. Moyano, S. García, J. Alcalá-Fdez, J. Luengo, A. Fernández, M. J. del Jesús, L. Sánchez, and F. Herrera, "KEEL 3.0: An open source software for multi-stage analysis in data mining," *J. Comput. Intell. Syst.*, vol. 10, pp. 1238–1249, Sep. 2017.
[45] N. Guenther and M. Schonlau, "Support vector machines," *Stata J.*, vol. 16, no. 4, pp. 917–937, 2016.
[46] M. A. Escalona-Morán, M. C. Soriano, I. Fischer, and C. R. Mirasso, "Electrocardiogram classification using reservoir computing with logistic regression," *IEEE J. Biomed. Health Inform.*, vol. 19, no. 3, pp. 892–898, May 2015.
[47] J.-S. Lee, "AUC4.5: AUC-based C4.5 decision tree algorithm for imbalanced data classification," *IEEE Access*, vol. 7, pp. 106034–106042, 2019.
[48] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, Jan. 2006.
[49] S. García, A. Fernández, J. Luengo, and F. Herrera, "Advanced non-parametric tests for multiple comparisons in the design of experiments in computational intelligence and data mining: Experimental analysis of power," *Inf. Sci.*, vol. 180, pp. 2044–2064, May 2010.
[50] S. García, J. Derrac, I. Triguero, C. J. Carmona, and F. Herrera, "Evolutionary-based selection of generalized instances for imbalanced classification," *Knowl-Based Syst.*, vol. 25, no. 1, pp. 3–12, Feb. 2012.

KE CHENG was born in Chizhou, Anhui, China, in 1972. He received the B.S. degree in mining engineering from the Hunan University of Science and Technology, Xiangtan, China, in 1996, the M.S. degree in agricultural process engineering from Jiangsu University, Zhenjiang, China, in 1999, and the Ph.D. degree in computer science from the Nanjing University of Science and Technology, Nanjing, China, in 2006.

Since 2008, he has been an Associate Professor with the School of Computer, Jiangsu University of Science and Technology, Zhenjiang, China. From 2002 to 2003, he was a Senior Visiting Scholar with the School of Computer Science, Southeast University, Nanjing. He has authored or coauthored more than 20 research articles. His research interests include machine learning, data mining, and bioinformatics.

CHEN ZHANG was born in Chizhou, Anhui, China, in 1997. He received the B.S. degree in automotive service engineering from Qingdao Technological University, Qingdao, China, in 2017. He is currently pursuing the master's degree in software engineering with the Jiangsu University of Science and Technology, Zhenjiang, China.

His research interests include big data analysis and artificial intelligence.

HUALONG YU was born in Harbin, China, in 1982. He received the B.S. degree in computer science from Heilongjiang University, Harbin, China, in 2005, and the M.S. and Ph.D. degrees in computer science from Harbin Engineering University, Harbin, in 2008 and 2010, respectively.

Since 2010, he has been an Associate Professor with the School of Computer, Jiangsu University of Science and Technology, Zhenjiang, China. From 2013 to 2017, he was a Postdoctoral Fellow with the School of Automation, Southeast University, Nanjing, China. From 2017 to 2018, he was a Senior Visiting Scholar with the Faculty of Information Technology, Monash University, Melbourne, Australia. He has authored or coauthored more than 70 journals and conference papers, and four monographs, including publications on the IEEE TNNLS, IEEE TFS, IEEE TCBB, IEEE Access, *Information Science*, KBS, and *Neurocomputing*. His research interests include machine learning, data mining, and bioinformatics.

Dr. Yu is also a member of the organizing committee of several international conferences, ACM, the China Computer Federation (CCF), and the Youth Committee of the Chinese Association of Automation (CAA). He is also an Associate Editor of IEEE Access, and an Active Reviewer for more than 20 high-quality international journals, including the IEEE TNNLS, TCYB, TKDE, TCBB, and ACM TKDD.

XIBEI YANG received the B.S. degree in computer science from Xuzhou Normal University, Xuzhou, China, in 2002, the M.S. degree in computer applications from the Jiangsu University of Science and Technology, Zhenjiang, China, in 2006 and the Ph.D. degree in pattern recognition and intelligence system from the Nanjing University of Science and Technology, Nanjing, China, in 2010.

Since 2018, he has been a Professor with the School of Computer, Jiangsu University of Science and Technology, Zhenjiang, China. He has published over 100 research articles on the professional journals and conferences. His research interests include granular computing and rough set theory.

Dr. Yang is also a member in the organizing committee of several international conferences. He is also a Reviewer for over ten high-quality international journals.

HAITAO ZOU received the B.S. degree in information security from Nankai University, Tianjin, China, in 2007, and the M.S. and Ph.D. degree in software engineering from the University of Macau, Macau, China, in 2010 and 2015, respectively.

He is currently a Lecturer with the School of Computer, Jiangsu University of Science and Technology, Zhenjiang, China. He has authored and coauthored several top journal articles. His research interests include databases, web information retrieval, and web mining.

Dr. Zou is also a Reviewer for a couple of web mining conferences.

SHANG GAO received the B.S. degree in system engineering from Air Force Engineering University, Xi'an, China, in 1993, the M.S. degree in military equipment from Air-Force Engineering University, Xi'an in 1996, and the Ph.D. degree in pattern recognition and intelligent systems from the Nanjing University of Science and Technology, Nanjing, China, in 2006, respectively.

Since 2009, he has been a Professor with the School of Computer, Jiangsu University of Science and Technology, Zhenjiang, China, where he has been the Dean, since 2017. He has published over 80 research articles on the professional journals and conferences. His research interests include optimization theory, swarm intelligence and machine learning.